Algorithms for DNA Sequencing

Today's goal • 2/3 items

Module 1
DNA sequencing, strings and matching

Welcome

Welcome to Algorithms for DNA Sequencing
Reading • 10 min

Pre Course Survey
Reading • 10 min

Syllabus
Reading • 10 min

Setting up Python (and Jupyter)
Reading • 10 min

Getting slides and notebooks
Reading • 10 min

Using data files with Python programs
Reading • 10 min

Module 1: DNA sequencing, strings and matching

Module 1 Introduction
Video • 1 min

Lecture: Why study this?
Video • 4 min

Lecture: DNA sequencing past and present
Video • 3 min

Lecture: Genomes as strings, reads as substrings
Video • 5 min

Lecture: String definitions and Python examples
Video • 3 min

Practical: String basics
Video • 7 min

Practical: Manipulating DNA strings
Video • 7 min

Practical: Downloading and parsing a genome
Video • 6 min

Lecture: How DNA gets copied
Video • 3 min

Optional lecture: How second-generation sequencers work
Video • 7 min

Optional lecture: Sequencing errors and base qualities
Video • 6 min

Lecture: Sequencing reads in FASTQ format
Video • 4 min

Practical: Working with sequencing reads
Video • 11 min

Practical: Analyzing reads by position
Video • 6 min

Lecture: Sequencers give pieces to genomic puzzles
Video • 5 min

Lecture: Read alignment and why it's hard
Video • 3 min

Lecture: Naive exact matching
Video • 10 min

Practical: Matching artificial reads
Video • 6 min

Practical: Matching real reads
Video • 7 min

Quiz

Module 1
Graded Assignment • 30 min

Programming Homework

Programming Homework 1 Instructions (Read First)
Reading • 10 min

Programming Homework 1
Graded Assignment • 30 min

Module 2
Preprocessing, indexing and approximate matching

Module 3
Edit distance, assembly, overlaps

Module 4
Algorithms for assembly

Setting up Python (and Jupyter)

Practice with Lab Sandbox

0:05 / 1:59 2x

This course uses Python. Examples in lectures are written in Python, the practical sessions use Python, and the programming assignments involve building on given Python code. All given Python code should be compatible with Python versions 2 and 3.

We will sometimes refer to something called "Jupyter" (which used to be called "IPython"). What is Jupyter? As explained in the Jupyter documentation, it's a convenient way to work interactively with Python code. It's similar to using Python in "interactive shell" mode but with many more features. We used Jupyter when building this course. We use it in the practical videos, and we used it to make notebooks like these. That said, *you do not have to use Jupyter for this course*. If you would like to, please do. But if you'd rather stick with Python, that's fine too.

How should you set up your Python environment to make it easy to work with the Python code given in this class? Below are two suggestions. Also, there is a course forum called "Setting up Python/Jupyter" dedicated to the topic of setting up your Python / Jupyter environment.

1. Install the Anaconda Python distribution. If you don't want to use Jupyter, you're done. If you do want to use Jupyter, then install Jupyter at the command line using "conda install jupyter" or "conda update jupyter" as described here.
2. Use wakari.io website. This enables you to use both Python and Jupyter at the same time. **Read the note** on the wakari.io website that says "Free usage of Wakari is intended for trial and occasional usage..." You probably don't want to use that website if you are developing programs that take a long time to run.

Note: there is also a website called try.jupyter.org that is similar to wakari.io. However, students in this course have had trouble with it in the past. For example, it seems that host can't resolve DNS names, which makes it hard to use data files provided in the programming assignments.

Best,

Ben

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →